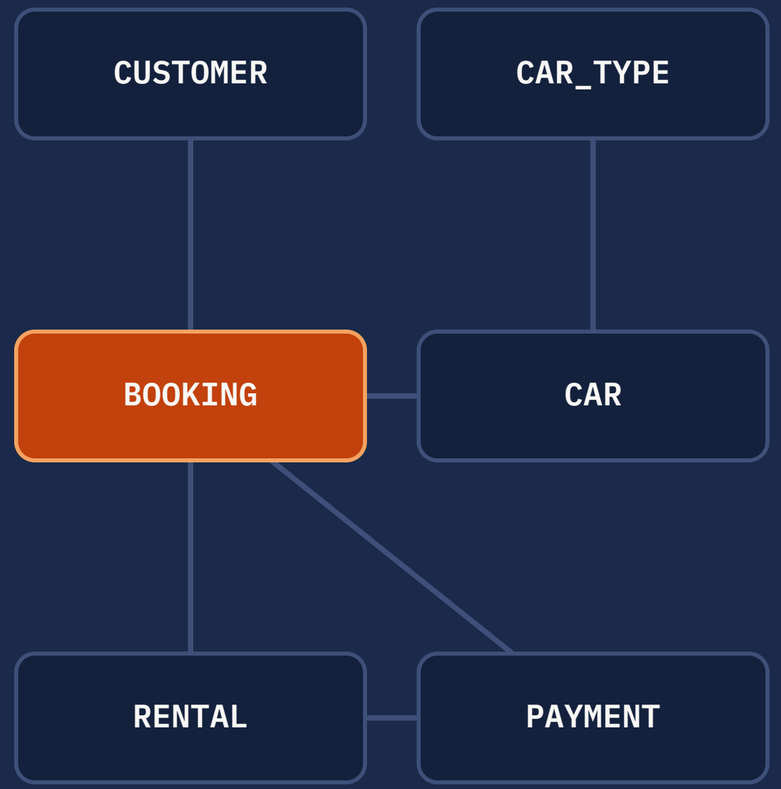
06066300 รายวิชาแนวคิดระบบฐานข้อมูล

ระบบเช่ารถยนต์ออนไลน์

Online Car Rental System

การออกแบบและพัฒนาฐานข้อมูลเชิงสัมพันธ์ ตั้งแต่ความต้องการของผู้ใช้จนถึงการทดสอบด้วย SQL

เสนอ ผศ.ดร.พัฒนพงษ์ ฉันทมิตรโอภาส
คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง



โครงสร้างฐานข้อมูล 6 ตาราง โดยมี BOOKING เป็นศูนย์กลาง

บทที่ 1 บทนำ

ที่มาและความสำคัญของปัญหา

การให้บริการเช่ารถยนต์เกี่ยวข้องกับข้อมูลหลายส่วนที่สัมพันธ์กัน หากขาดระบบฐานข้อมูลที่เหมาะสม จะเกิดปัญหาดังนี้

การจัดเก็บแบบเดิม

ข้อมูลถูกบันทึกในเอกสาร และกระจายอยู่หลายแหล่ง ทำให้สืบค้นได้ล่าช้า

ข้อมูลซ้ำซ้อน

ข้อมูลลูกค้า รถยนต์ และอัตราค่าเช่า ถูกบันทึกซ้ำ จนเกิดความคลาดเคลื่อน

การจองซ้อน

รถยนต์คันเดียวกัน อาจถูกจองในช่วงเวลาที่ทับซ้อนกัน (Double Booking)

ภาระการติดต่อ

ลูกค้าไม่สามารถตรวจสอบสถานะรถได้ด้วยตนเอง ต้องติดต่อสอบถามเป็นรายกรณี

แนวทางแก้ไข พัฒนาฐานข้อมูลกลางที่รวบรวมข้อมูลลูกค้า รถยนต์ การจอง การเช่า และการชำระเงินไว้ในระบบเดียว เพื่อให้ตรวจสอบสถานะได้ทันที และป้องกันการจองซ้อนโดยอัตโนมัติ

วัตถุประสงค์และขอบเขตของระบบ

วัตถุประสงค์

1. เพื่อศึกษาและวิเคราะห์กระบวนการให้บริการเช่ารถยนต์ และนำหลักการออกแบบฐานข้อมูลมาประยุกต์ใช้
2. เพื่อออกแบบฐานข้อมูลที่สามารถจัดเก็บ สืบค้น ตรวจสอบ และปรับปรุงข้อมูลได้อย่างเป็นระบบ
3. เพื่อลดความซ้ำซ้อนและความคลาดเคลื่อนในการจัดการรถยนต์ การจอง การรับรถ และการคืนรถ

ขอบเขตของระบบ

ผู้ใช้บริการ (Customer)
ลงทะเบียน ค้นหารถยนต์ ทำรายการจอง ชำระเงิน และติดตามสถานะ

ผู้ดูแลระบบ (Admin)
บริหารข้อมูลลูกค้า รถยนต์ ประเภทรถ อัตราค่าเช่า การจอง การรับคืนรถ และการชำระเงิน

ฐานข้อมูลประกอบด้วย 6 ตาราง ได้แก่ **CUSTOMER, CAR_TYPE, CAR, BOOKING, RENTAL** และ **PAYMENT**

หลักการที่ใช้ในการออกแบบฐานข้อมูล

การออกแบบอาศัยหลักการสามส่วน ตั้งแต่ระดับแนวคิดจนถึงการกำหนดโครงสร้างข้อมูลจริง

01

Entity Relationship Diagram

แบบจำลองระดับแนวคิด ที่แสดงเอนทิตี แอตทริบิวต์ และความสัมพันธ์ระหว่างข้อมูล โดยไม่ขึ้นกับ DBMS

Strong และ Weak Entity, Cardinality 1:1, 1:N, M:N

02

Normalization

กระบวนการปรับโครงสร้างตาราง เพื่อลดความซ้ำซ้อน และขจัดความผิดปกติในการจัดการข้อมูล

Insertion, Update และ Deletion Anomaly

03

Data Dictionary

เอกสารอ้างอิงกลาง ที่อธิบายโครงสร้าง ชนิด และข้อจำกัดของข้อมูล ให้ผู้พัฒนาเข้าใจตรงกัน

Metadata และ Data Integrity

ความต้องการของผู้ใช้ (User Requirements)

ความต้องการด้านฟังก์ชัน

ระบบลูกค้า	เพิ่ม แก้ไข ค้นหา และลบข้อมูลลูกค้า พร้อมตรวจสอบเลขใบอนุญาตขับขี่
ระบบรถยนต์	จัดหมวดหมู่รถเป็น Sedan, SUV, Van และ Eco Car พร้อมปรับปรุงสถานะรถ
ระบบการเช่า	กำหนดวันรับและวันคืนรถ และคำนวณค่าเช่ารวมจากจำนวนวันคูณค่าเช่าต่อวัน
ระบบชำระเงิน	รองรับการชำระหลายงวดต่อการเช่าหนึ่งครั้ง ได้แก่ เงินมัดจำ ค่าเช่า และค่าปรับ

ความต้องการที่ไม่ใช่ฟังก์ชัน

ความปลอดภัย

ข้อมูลส่วนบุคคลและใบขับขี่ต้องได้รับการเข้ารหัสและจัดเก็บอย่างปลอดภัย

ความถูกต้องของข้อมูล

ต้องป้องกันการจองรถคันเดียวกันในช่วงเวลาที่ทับซ้อนกัน

การเลือกใช้ซอฟต์แวร์และสถาปัตยกรรมระบบ

PostgreSQL บน Supabase

เหตุผลในการเลือกใช้

- ข้อมูลมีความสัมพันธ์ชัดเจน จึงเหมาะกับฐานข้อมูลเชิงสัมพันธ์
- รองรับ EXCLUDE Constraint สำหรับป้องกันการจองซ้อนได้ในตัว
- รองรับ Transaction แบบ ACID, CTE และ Window Function
- Supabase ให้บริการบนคลาวด์ ใช้งานได้ตลอด 24 ชั่วโมง

สถาปัตยกรรมแบบสามชั้น (3-Tier Architecture)

ส่วนแสดงผล (Frontend)

React หรือ Flutter ติดตั้งบน Vercel

ส่วนประมวลผล (Backend API)

RESTful API พัฒนาด้วย FastAPI หรือ Node.js

ส่วนจัดการฐานข้อมูล (Database)

ออกแบบสำหรับ Supabase ทดสอบบน PostgreSQL 16

ลักษณะการทำงานของระบบ

กระบวนการตั้งแต่ลงทะเบียนจนถึงคืนรถแบ่งเป็น 5 ขั้นตอน แต่ละขั้นตอนอ่านหรือบันทึกข้อมูลในตารางที่กำหนด

1

ลงทะเบียน

บันทึกข้อมูลผู้ใช้บริการและตรวจสอบใบขับขี่

`CUSTOMER`

2

ค้นหารถยนต์

แสดงเฉพาะรถยนต์ ที่มีสถานะว่าง (Available)

`CAR`
`CAR_TYPE`

3

จองและเช่า

คำนวณค่าเช่ารวม จากจำนวนวัน คูณอัตราต่อวัน และเปลี่ยนสถานะรถ เป็น Rented

`BOOKING`
`RENTAL`

4

ชำระเงิน

บันทึกรายการชำระเงินและยืนยันการจอง (Confirmed)

`PAYMENT`

5

คืนรถ

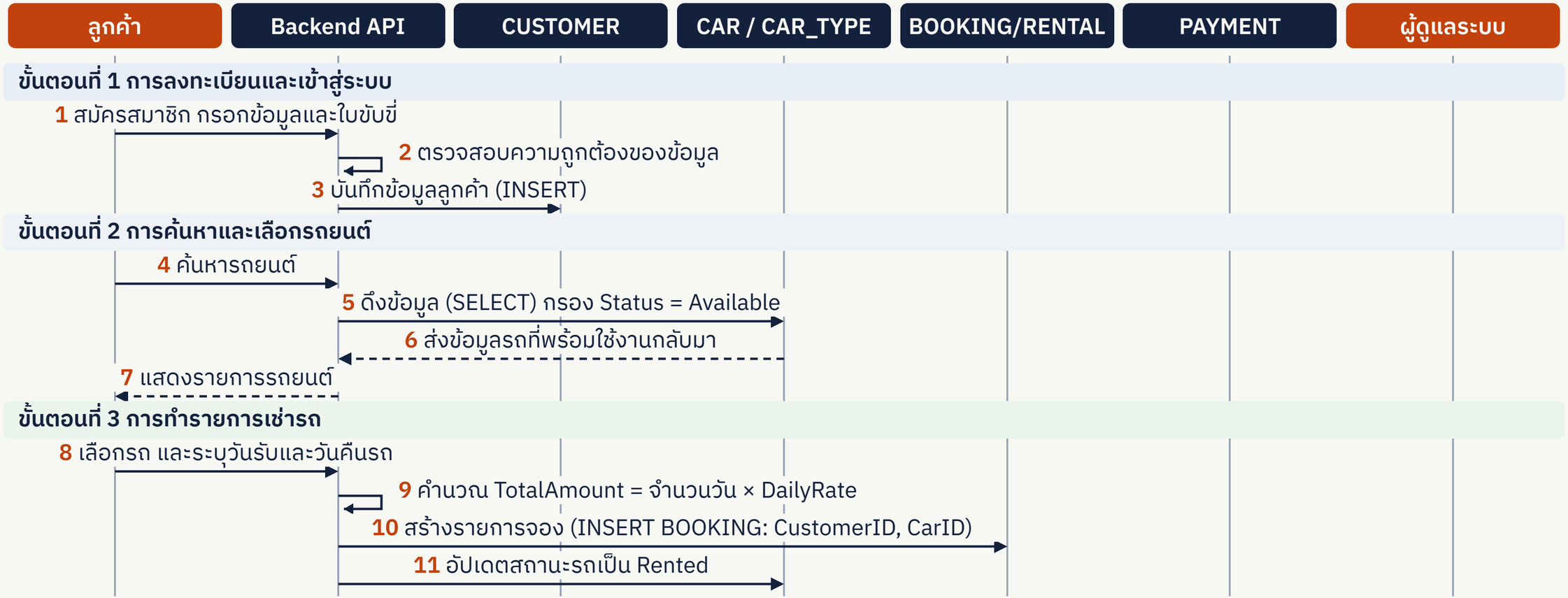
ผู้ดูแลระบบ ตรวจสภาพรถ ปิดรายการเช่า และคืนสถานะรถเป็นว่าง

`RENTAL`
`CAR`

สถานะรถ `Available` → `Rented` → `Available`

สถานะการจอง `Pending` → `Confirmed` → `Completed`

หลักการทำงานของระบบ (Sequence Diagram) ส่วนที่ 1/2



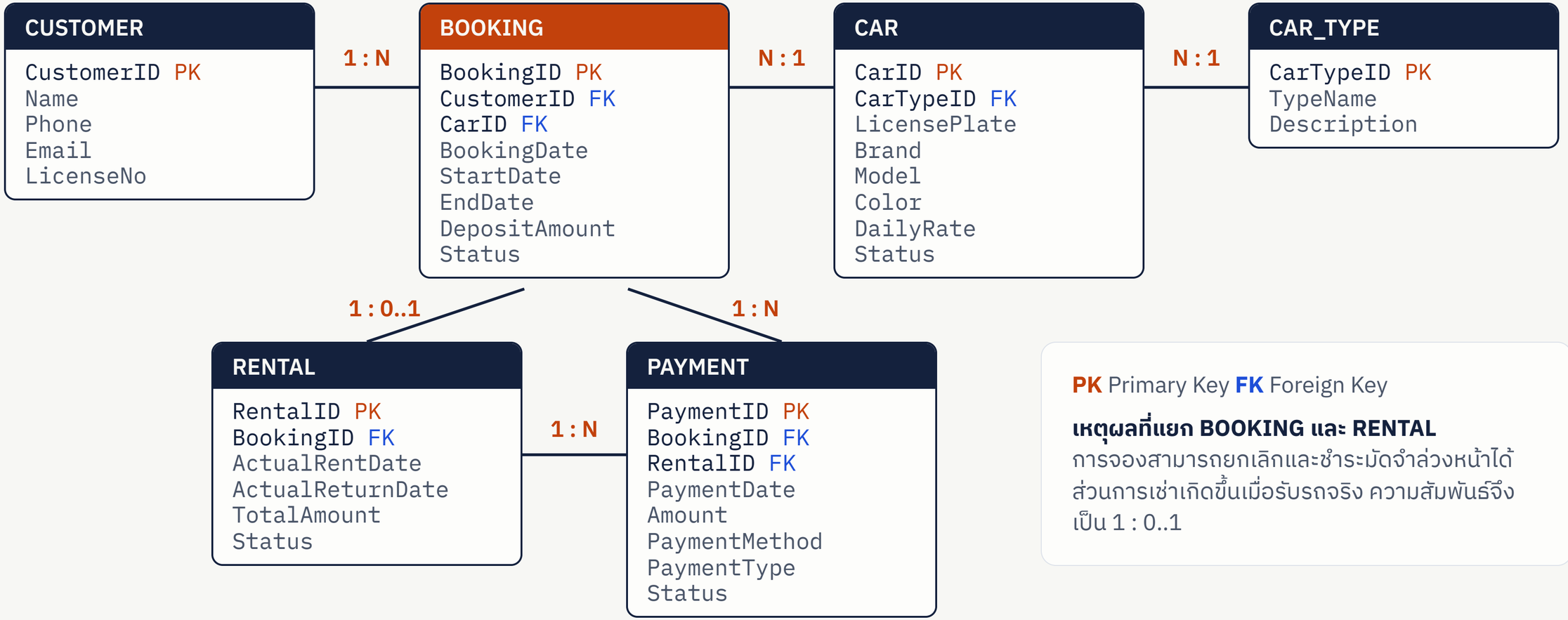
หลักการทำงานของระบบ (Sequence Diagram) ส่วนที่ 2/2



กฎทางธุรกิจและการบังคับใช้ในฐานข้อมูล

กฎ	รายละเอียด	วิธีบังคับใช้
BR1	ลูกค้าหนึ่งคนทำรายการเช่าได้หลายรายการ แต่แต่ละรายการเป็นของลูกค้าเพียงคนเดียว	Foreign Key แบบ 1 ต่อ N
BR2	รถยนต์หนึ่งคันไม่สามารถถูกจองซ้อนในช่วงเวลาเดียวกันได้	EXCLUDE Constraint
BR3	รถยนต์แต่ละคันอยู่ในประเภทเดียว และประเภทหนึ่งมีรถได้หลายคัน	Foreign Key ที่ห้ามเป็นค่าว่าง
BR4	รายการเช่าต้องระบุรถยนต์ วันที่เช่า และวันที่คืน	NOT NULL
BR5	รายการเช่าหนึ่งรายการชำระเงินได้หลายครั้ง เช่น มัดจำและส่วนที่เหลือ	ตาราง PAYMENT แบบ N ต่อ 1
BR6	สร้างรายการเช่าได้เฉพาะรถยนต์ที่มีสถานะว่าง	ตรวจสอบในระดับแอปพลิเคชัน (คำสั่ง E1 กรองสถานะ Available)
BR7	วันที่คืนรถต้องอยู่หลังวันที่เช่า	CHECK Constraint

Entity Relationship Diagram (ERD)

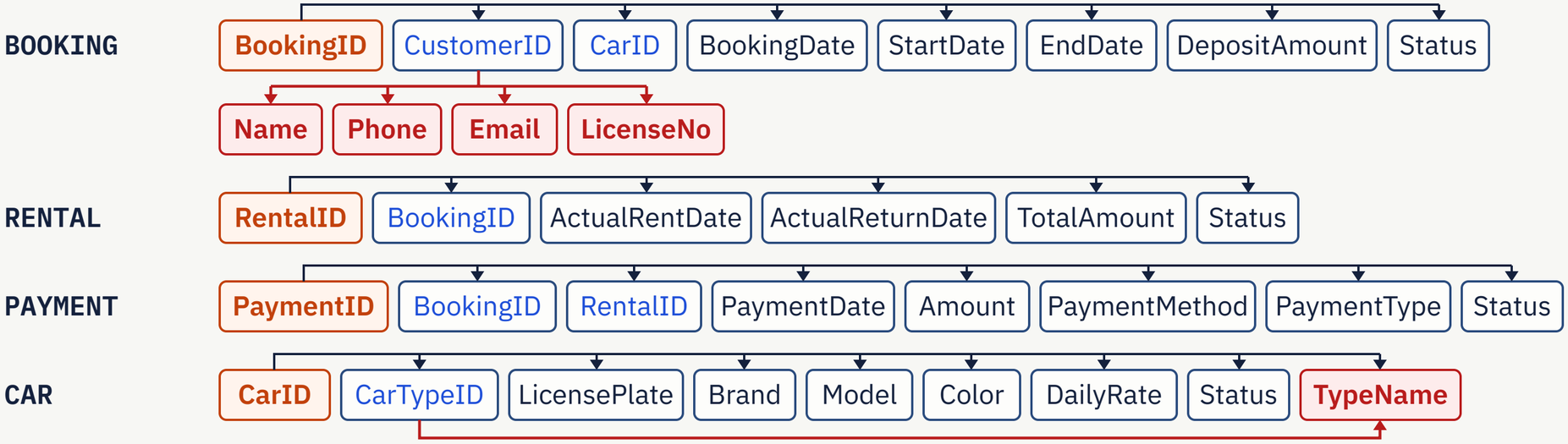


PK Primary Key **FK** Foreign Key

เหตุผลที่แยก BOOKING และ RENTAL
การจองสามารถยกเลิกและชำระมัดจำล่วงหน้าได้ ส่วนการเช่าเกิดขึ้นเมื่อรับรถจริง ความสัมพันธ์จึงเป็น 1 : 0..1

First Normal Form (1NF)

PK คีย์หลัก **FK** คีย์นอก **ช่องสีแดง** แอตทริบิวต์ที่ต้องแยกออก
เส้นดำ ขึ้นกับคีย์หลัก **เส้นแดง** ขึ้นกับแอตทริบิวต์ที่ไม่ใช่คีย์



เงื่อนไข 1NF

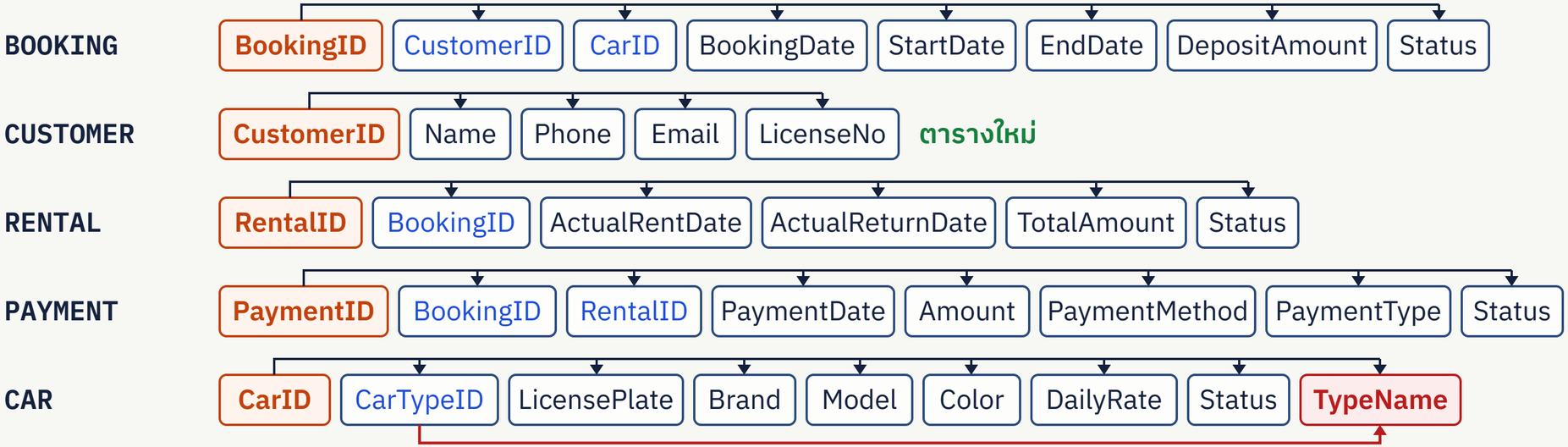
ทุกแอตทริบิวต์เก็บค่าเพียงค่าเดียว (Atomic) ไม่มีกลุ่มข้อมูลซ้ำ และทุกตารางมีคีย์หลัก

ปัญหาที่พบ

BOOKING เก็บข้อมูลลูกค้าซ้ำทุกครั้งที่จอง เพราะ `CustomerID → Name, Phone, Email, LicenseNo` และ CAR เก็บ TypeName ซ้ำ เพราะ `CarTypeID → TypeName`

Second Normal Form (2NF)

PK คีย์หลัก **FK** คีย์นอก **ช่องสีแดง** แอตทริบิวต์ที่ต้องแยกออก
เส้นดำ ขึ้นกับคีย์หลัก **เส้นแดง** ขึ้นกับแอตทริบิวต์ที่ไม่ใช่คีย์



เงื่อนไข 2NF

อยู่ในรูป 1NF และแอตทริบิวต์ที่ไม่ใช่คีย์ทุกตัว ต้องขึ้นกับคีย์หลักทั้งหมดของตาราง

การดำเนินการ

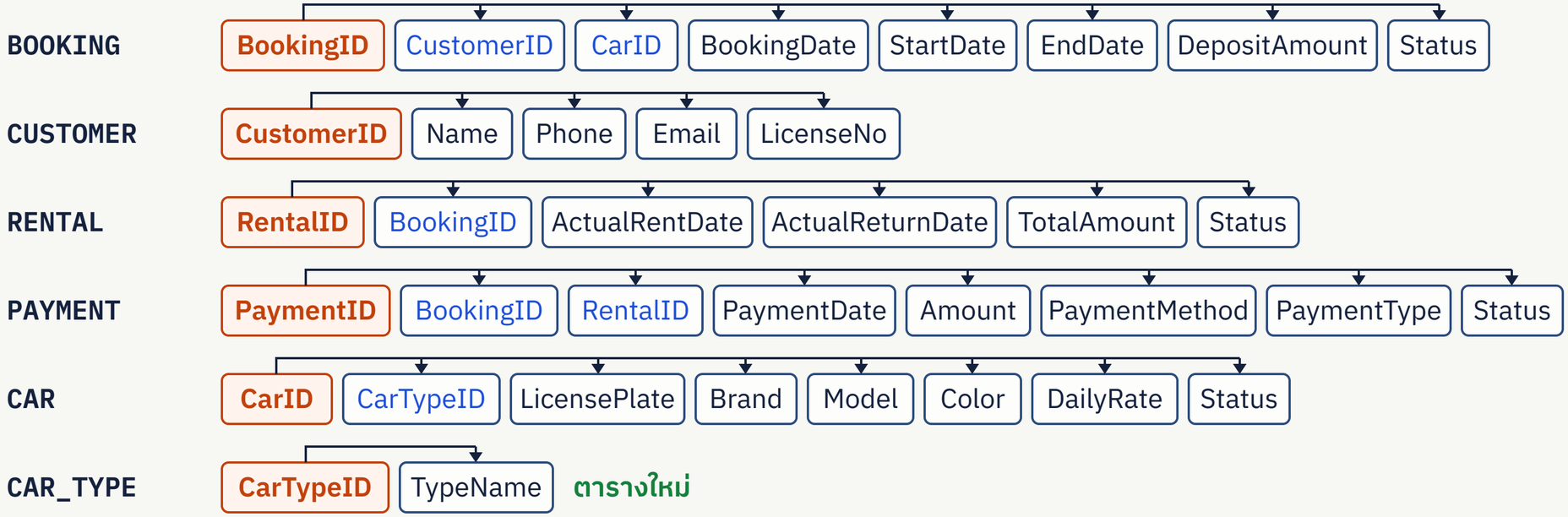
แยก Name, Phone, Email, LicenseNo ออกเป็นตาราง CUSTOMER และให้ BOOKING อ้างอิงผ่านคีย์นอก CustomerID

สิ่งที่ยังเหลือ

ตาราง CAR ยังเก็บ TypeName ซึ่งขึ้นกับ CarTypeID จึงต้องแก้ไขในขั้น 3NF

Third Normal Form (3NF)

PK คีย์หลัก **FK** คีย์นอก **ช่องสีแดง** แอตทริบิวต์ที่ต้องแยกออก
เส้นดำ ขึ้นกับคีย์หลัก **เส้นแดง** ขึ้นกับแอตทริบิวต์ที่ไม่ใช่คีย์



เงื่อนไข 3NF

อยู่ในรูป 2NF และไม่มี Transitive Dependency

การดำเนินการ

แยก TypeName ออกเป็นตาราง CAR_TYPE เพราะ `CarID → CarTypeID → TypeName`

ผลลัพธ์

ได้ 6 ตาราง ทุกแอตทริบิวต์ขึ้นกับคีย์หลักเท่านั้น ไม่มีเส้นแดงเหลืออยู่

พจนานุกรมข้อมูล (Data Dictionary) ส่วนที่ 1/3

CUSTOMER ตารางลูกค้า

Field	Data Type	Key	คำอธิบาย
CustomerID	INT	PK	รหัสลูกค้า
Name	VARCHAR(100)		ชื่อลูกค้า
Phone	VARCHAR(20)		เบอร์โทรศัพท์
Email	VARCHAR(100)		อีเมลของลูกค้า
LicenseNo	VARCHAR(20)		เลขที่ใบขับขี่

PK: CustomerID

CAR_TYPE ตารางประเภทรถ

Field	Data Type	Key	คำอธิบาย
CarTypeID	INT	PK	รหัสประเภทรถ
TypeName	VARCHAR(50)		ชื่อประเภทรถ
Description	VARCHAR(255)		รายละเอียดของประเภทรถ

PK: CarTypeID

TypeName เช่น Sedan, SUV, Van, Eco Car

พจนานุกรมข้อมูล (Data Dictionary) ส่วนที่ 2/3

CAR ตารางรถยนต์

Field	Data Type	Key	คำอธิบาย
CarID	INT	PK	รหัสรถยนต์
CarTypeID	INT	FK	รหัสประเภทรถ
LicensePlate	VARCHAR(20)		หมายเลขทะเบียนรถ
Brand	VARCHAR(50)		ยี่ห้อรถ
Model	VARCHAR(50)		รุ่นรถ
Color	VARCHAR(30)		สีรถ
DailyRate	DECIMAL(10,2)		ราคาเช่าต่อวัน
Status	VARCHAR(20)		สถานะรถ

PK: CarID FK: CarTypeID → CAR_TYPE

BOOKING ตารางการจอง

Field	Data Type	Key	คำอธิบาย
BookingID	INT	PK	รหัสการจอง
CustomerID	INT	FK	รหัสลูกค้าที่จอง
CarID	INT	FK	รหัสรถที่จอง
BookingDate	TIMESTAMP		วันที่ทำการจอง
StartDate	TIMESTAMP		วันเริ่มเช่า
EndDate	TIMESTAMP		วันสิ้นสุดการเช่า
DepositAmount	DECIMAL(10,2)		จำนวนเงินมัดจำ
Status	VARCHAR(20)		สถานะการจอง

PK: BookingID FK: CustomerID → CUSTOMER, CarID → CAR

พจนานุกรมข้อมูล (Data Dictionary) ส่วนที่ 3/3

RENTAL ตารางรายการเช่า

Field	Data Type	Key	คำอธิบาย
RentalID	INT	PK	รหัสรายการเช่า
BookingID	INT	FK	รหัสการจองที่เกี่ยวข้อง
ActualRentDate	TIMESTAMP		วันที่รับรถจริง
ActualReturnDate	TIMESTAMP		วันที่คืนรถจริง
TotalAmount	DECIMAL(10,2)		ค่าเช่ารวมทั้งหมด
Status	VARCHAR(20)		สถานะการเช่า

PK: RentalID FK: BookingID → BOOKING

ค่าที่กำหนดในตาราง PAYMENT ได้แก่ PaymentMethod คือ Cash, Credit Card, QR และ PaymentType คือ Deposit, Rental Fee, Late Fee

หมายเหตุ ในสคริปต์ SQL ใช้ชื่อแบบ snake_case เช่น customer_id คีย์หลักใช้ชนิด SERIAL และ DATETIME ใช้ TIMESTAMP ตาม PostgreSQL

PAYMENT ตารางการชำระเงิน

Field	Data Type	Key	คำอธิบาย
PaymentID	INT	PK	รหัสการชำระเงิน
BookingID	INT	FK	รหัสการจอง
RentalID	INT	FK	รหัสรายการเช่า
PaymentDate	TIMESTAMP		วันที่ชำระเงิน
Amount	DECIMAL(10,2)		จำนวนเงินที่ชำระ
PaymentMethod	VARCHAR(20)		วิธีชำระเงิน
PaymentType	VARCHAR(20)		ประเภทการชำระ
Status	VARCHAR(20)		สถานะการชำระเงิน

PK: PaymentID FK: BookingID → BOOKING, RentalID → RENTAL

ขั้นตอนการสร้างฐานข้อมูล

1

เตรียมระบบ

สร้างฐานข้อมูลบน PostgreSQL 16 และเปิดใช้ส่วนขยาย btree_gist

2

ตารางหลัก

CUSTOMER และ CAR_TYPE ซึ่งไม่มีคีย์นอก

3

ตารางอ้างอิง

CAR, BOOKING, RENTAL และ PAYMENT ตามลำดับคีย์นอก

4

สร้างดัชนี

ดัชนี 3 รายการ เพื่อเร่งการค้นหาสถานะรถและการจอง

5

เพิ่มข้อมูลตัวอย่าง

INSERT ข้อมูล 44 แถว ครอบคลุมทุกสถานะ

6

ทดสอบ

คำสั่งทดสอบ 19 คำสั่ง แบ่งเป็น 3 ระดับ

ลำดับการสร้างตาราง (ตารางที่ถูกอ้างอิงต้องสร้างก่อน)

CUSTOMER, CAR_TYPE → CAR → BOOKING → RENTAL → PAYMENT

คำสั่งในขั้นตอนที่ 1 และ 4

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE INDEX idx_car_status ON car(status);
```

สร้างฐานข้อมูลด้วย SQL DDL

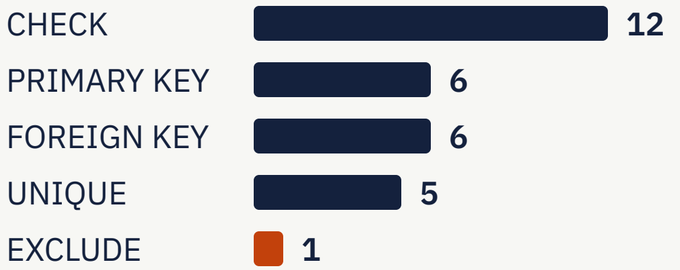
```sql
CREATE TABLE booking (
  booking_id  SERIAL PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES customer,
  car_id      INT NOT NULL REFERENCES car,
  -- booking_date, deposit_amount (ละไว้)
  start_date  TIMESTAMP NOT NULL,
  end_date    TIMESTAMP NOT NULL,
  status VARCHAR(20) DEFAULT 'Pending' CHECK
    (status IN ('Pending', 'Confirmed',
                'Cancelled', 'Completed')),
  CONSTRAINT chk_booking_dates        -- BR7
    CHECK (end_date > start_date),
  CONSTRAINT no_overlap EXCLUDE USING gist (  -- BR2
    car_id WITH =, tsrange(start_date, end_date) WITH &&)
    WHERE (status <> 'Cancelled')
);
```

PRIMARY / FOREIGN KEY
เชื่อมโยงการจองกับลูกค้าและรถยนต์

CHECK
จำกัดค่าสถานะ และกำหนดให้วันคืนอยู่หลังวันรับ (BR7)

EXCLUDE
ปฏิเสธการจองรถคันเดิมในช่วงเวลาที่ทับซ้อน (BR2)

Constraints ทั้งระบบ 30 รายการ ใน 6 ตาราง



ตัวอย่างข้อมูล (Sample Data)

4	8	6	8	6	12
ประเภทรถ	รถยนต์	ลูกค้า	การจอง	การเช่า	การชำระเงิน

รถ	ประเภท	บาท/วัน	Status
Toyota Camry	Sedan	1,800	Available
Honda Civic	Sedan	1,500	Rented
Toyota Fortuner	SUV	2,500	Available
Honda CR-V	SUV	2,300	Rented
Toyota Commuter	Van	3,000	Available
Hyundai H-1	Van	2,800	Maintenance
Nissan Almera	Eco Car	900	Available
Toyota Yaris Ativ	Eco Car	950	Available

การออกแบบข้อมูลทดสอบ

- ครอบคลุมสถานะรถครบทั้ง 3 แบบ
- ครอบคลุมสถานะการจองทั้ง 4 แบบ รวมถึงการยกเลิก
- มีการชำระหลายงวด ได้แก่ มัดจำ ค่าเช่า และค่าปรับ
- มีการเช่าที่ยังไม่คืนรถ สำหรับทดสอบยอดค้างชำระ

การทดสอบระดับง่าย (Easy) จำนวน 6 คำสั่ง

```sql
-- E1 รถที่ว่าง เรียงตามราคา
SELECT car_id, brand, model, daily_rate FROM car
WHERE status = 'Available' ORDER BY daily_rate;
-- E2 ค้นหาลูกค้าจากเลขใบขับขี่
SELECT * FROM customer WHERE license_no = 'DL-10003';
-- E3 รถที่ค่าเช่าไม่เกิน 2,000 บาทต่อวัน
SELECT brand, model, daily_rate FROM car
WHERE daily_rate <= 2000 ORDER BY daily_rate DESC;
-- E4 เพิ่มลูกค้าใหม่
INSERT INTO customer (name, phone, email, license_no)
VALUES ('ณัฐวุฒิ ทองดี', '087-777-7777',
        'nattawut@mail.com', 'DL-10007');
-- E5 ปรับสถานะรถที่ซ่อมเสร็จ
UPDATE car SET status = 'Available'
WHERE license_plate = 'นค 1122';
-- E6 ลบลูกค้าที่ยังไม่เคยจอง
DELETE FROM customer WHERE license_no = 'DL-10007';
```

ผลการทดสอบ ผ่านทั้ง 6 คำสั่ง

- **E1 SELECT WHERE ORDER BY**
  5 แถว จาก Almera 900 บาท ถึง Commuter 3,000 บาท
- **E2 SELECT WHERE**
  1 แถว คือ ธนากร มั่นคง
- **E3 WHERE ช่วงราคา**
  4 แถว ได้แก่ Camry, Civic, Yaris Ativ และ Almera
- **E4 INSERT**
  INSERT 0 1 เพิ่มลูกค้าสำเร็จ
- **E5 UPDATE**
  UPDATE 1 รถ H-1 กลับมาว่าง
- **E6 DELETE**
  DELETE 1 ลบลูกค้าที่เพิ่มใน E4

การทดสอบระดับปานกลาง (Medium) ส่วนที่ 1/3

M1 **JOIN 3 ตาราง** รายการจองพร้อมชื่อลูกค้าและรถ

```sql
SELECT b.booking_id, c.name,
  ca.brand || ' ' || ca.model AS car, b.status
FROM booking b JOIN customer c USING (customer_id)
  JOIN car ca USING (car_id) ORDER BY b.start_date;
```

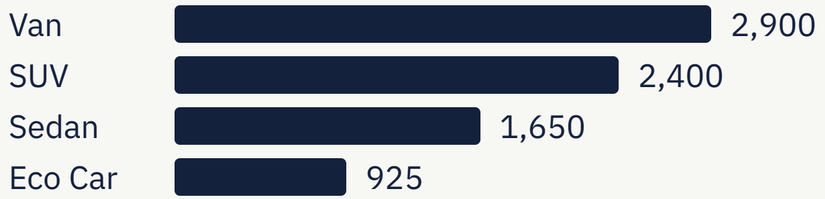
ผลลัพธ์ 8 แถว (แสดง 3 แถวแรก)

id	ลูกค้า	รถ
1	สมชาย ใจดี	Toyota Camry
2	สมหญิง รักษ์ดี	Toyota Fortuner
3	สมชาย ใจดี	Nissan Almera

M2 **LEFT JOIN GROUP BY AVG** ค่าเช่าเฉลี่ยต่อวันแยกตามประเภทรถ

```sql
SELECT t.type_name, ROUND(AVG(c.daily_rate), 2)
FROM car_type t LEFT JOIN car c USING (car_type_id)
GROUP BY t.type_name ORDER BY 2 DESC;
```

ผลลัพธ์ 4 แถว (บาทต่อวัน)



การทดสอบระดับปานกลาง (Medium) ส่วนที่ 2/3

M3 **SUM GROUP BY** ยอดรับเงินแยกตามช่องทางการชำระ

```
SELECT payment_method, COUNT(*), SUM(amount)
FROM payment WHERE status = 'Paid'
GROUP BY payment_method ORDER BY 3 DESC;
```

ผลลัพธ์ 3 แถว

ช่องทาง	รายการ	ยอดรวม
Credit Card	4	15,900
Cash	3	9,500
QR	4	3,800

M4 **GROUP BY HAVING** ลูกค้าที่จองมากกว่า 1 ครั้ง

```
SELECT c.name, COUNT(*) AS bookings
FROM customer c JOIN booking b USING (customer_id)
GROUP BY c.customer_id, c.name HAVING COUNT(*) > 1;
```

ผลลัพธ์ 2 แถว

ลูกค้า	จำนวนการจอง
สมหญิง รักษ์ดี	2
สมชาย ใจดี	2

การทดสอบระดับปานกลาง (Medium) ส่วนที่ 3/3

M5 **JOIN 4 ตาราง** รถที่อยู่ระหว่างการเช่าและวันกำหนดคืน

```
SELECT ca.license_plate, ca.model, cu.name,
       b.end_date::date AS due_date
FROM rental r JOIN booking b USING (booking_id)
  JOIN car ca USING (car_id)
  JOIN customer cu USING (customer_id)
WHERE r.status = 'Active';
```

ผลลัพธ์ 2 แถว

ทะเบียน	รุ่น	กำหนดคืน
กค 5678	Civic	15 ต.ค.
ขข 3456	CR-V	12 ต.ค.

M6 **CREATE VIEW (ทดสอบ DDL)** สร้าง View จากคำสั่ง M5 แล้วเรียกใช้

```
CREATE VIEW v_active_rentals AS
  SELECT ...; -- คำสั่งเดียวกับ M5
SELECT * FROM v_active_rentals ORDER BY due_date;
```

ผลลัพธ์ CREATE VIEW และ 2 แถว

View ถูกสร้างสำเร็จ และแสดงรถ 2 คันเช่นเดียวกับ M5 โดยเรียงตามวันกำหนดคืน

การทดสอบระดับซับซ้อน (Complex) ส่วนที่ 1/3

C1 CTE + LEFT JOIN ยอดค้างชำระของแต่ละการเช่า

```
WITH paid AS (
  SELECT booking_id, SUM(amount) AS paid
  FROM payment WHERE status = 'Paid' GROUP BY booking_id)
SELECT r.rental_id, cu.name, r.total_amount,
  r.total_amount - COALESCE(p.paid, 0) AS outstanding
FROM rental r JOIN booking b USING (booking_id)
  JOIN customer cu USING (customer_id)
  LEFT JOIN paid p USING (booking_id)
WHERE r.total_amount - COALESCE(p.paid, 0) > 0;
```

ผลลัพธ์ 2 แถว (ก่อนรันคำสั่ง C5)

id	ลูกค้า	ค้างชำระ
5	พิมพ์ชนก สุขใจ	9,500
6	วีรภัทร ศรีสุข	5,400

C2 Window Function RANK() จัดอันดับรถตามรายได้

```
SELECT ca.brand || ' ' || ca.model AS car,
  SUM(r.total_amount) AS revenue,
  RANK() OVER (ORDER BY SUM(r.total_amount) DESC)
FROM rental r JOIN booking b USING (booking_id)
  JOIN car ca USING (car_id) GROUP BY car;
```

ผลลัพธ์ 6 แถว (แสดง 3 อันดับแรก)

1	Honda Civic	10,500
2	Toyota Fortuner	10,000
3	Toyota Commuter	9,000

การทดสอบระดับซับซ้อน (Complex) ส่วนที่ 2/3

C3 NOT EXISTS + ช่วงเวลา

รถที่ว่างตลอดช่วงวันที่ 19 ถึง 22 ต.ค.

```
SELECT ca.brand, ca.model FROM car ca
WHERE ca.status <> 'Maintenance'
  AND NOT EXISTS (
    SELECT 1 FROM booking b
    WHERE b.car_id = ca.car_id
      AND b.status <> 'Cancelled'
      AND tsrange(b.start_date, b.end_date)
       && tsrange('2026-10-19', '2026-10-22'));
```

ผลลัพธ์ 7 แถว

ทุกคันยกเว้น Camry ซึ่งมีการจองวันที่ 20 ถึง 23 ต.ค. (รวม H-1 ที่กลับมาว่างหลังคำสั่ง E5)

C4 Subquery ใน HAVING

ลูกค้าที่ชำระเงินสูงกว่าค่าเฉลี่ยต่อลูกค้า

```
SELECT cu.name, SUM(p.amount) AS spent
FROM customer cu
  JOIN booking b USING (customer_id)
  JOIN payment p USING (booking_id)
WHERE p.status = 'Paid'
GROUP BY cu.customer_id, cu.name
HAVING SUM(p.amount) > (SELECT AVG(s)
  FROM (...ยอดรวมต่อลูกค้า...) x);
```

ผลลัพธ์ 3 แถว

1	สมหญิง รักษ์ดี	10,500
2	ธนากร มั่นคง	9,000
3	สมชาย ใจดี	7,200

การทดสอบระดับซับซ้อน (Complex) ส่วนที่ 3/3

C5 **Transaction (BEGIN ... COMMIT)** บันทึกการคืนรถของรายการเช่าที่ 6

```
BEGIN;
  UPDATE rental SET status = 'Completed',
    actual_return_date = '2026-10-12 08:45'
  WHERE rental_id = 6;
  INSERT INTO payment (booking_id, rental_id,
    amount, payment_method, payment_type)
  VALUES (6, 6, 5400, 'Credit Card', 'Rental Fee');
  UPDATE booking SET status = 'Completed'
  WHERE booking_id = 6;
  UPDATE car SET status = 'Available' WHERE car_id = 4;
COMMIT;
```

ผลลัพธ์ ทุกคำสั่งสำเร็จ และ COMMIT

RENTAL	Active	➡ **Completed**
PAYMENT	ค้างชำระ 5,400	➡ **ชำระครบ**
BOOKING	Confirmed	➡ **Completed**
CAR (CR-V)	Rented	➡ **Available**

หากคำสั่งใดล้มเหลว ระบบจะยกเลิกทั้งหมด (ROLLBACK) ข้อมูล 4 ตารางจึงสอดคล้องกันเสมอ ตามหลัก ACID

ทดสอบความถูกต้องของข้อมูล (Constraint Test)

C6 กฎ BR2 ห้ามจองซ้อน

จองรถ Camry วันที่ 21–24 ต.ค. ซึ่งทับซ้อนกับการจองที่ 7 (20–23 ต.ค.)

```
INSERT INTO booking (customer_id,
  car_id, start_date, end_date)
VALUES (3, 1, '2026-10-21', '2026-10-24');
```

⚠ ERROR: conflicting key value violates exclusion constraint "no_overlap"

C7 กฎ BR7 วันคืนต้องหลังวันรับ

กำหนดวันคืน 1 พ.ย. ก่อนวันรับ 5 พ.ย.

```
INSERT INTO booking (customer_id,
  car_id, start_date, end_date)
VALUES (3, 3, '2026-11-05', '2026-11-01');
```

⚠ ERROR: new row for relation "booking" violates check constraint "chk_booking_dates"

ผลการทดสอบ ฐานข้อมูลปฏิเสธข้อมูลที่ไม่ถูกต้องได้เอง โดยไม่ต้องพึ่งโปรแกรมประยุกต์

สรุปผลการดำเนินงาน

ประเมินผลตามวัตถุประสงค์

เพื่อศึกษาและวิเคราะห์กระบวนการให้บริการเช่ารถยนต์

วิเคราะห์ความต้องการ 4 ระบบย่อย กฎทางธุรกิจ 7 ข้อ และลำดับการทำงาน 5 ขั้นตอน

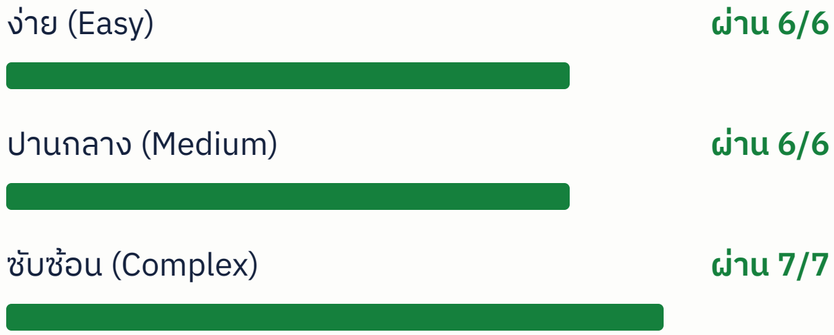
เพื่อออกแบบฐานข้อมูลที่จัดเก็บ สืบค้น และปรับปรุงข้อมูลได้อย่างเป็นระบบ

ได้ 6 ตารางในรูปแบบ 3NF พร้อม ERD และพจนานุกรมข้อมูล

เพื่อลดความซ้ำซ้อนและความคลาดเคลื่อนของข้อมูล

บังคับความถูกต้องด้วย 30 constraints และป้องกันการจองซ้อนในระดับฐานข้อมูล

ผลการทดสอบ SQL



19
คำสั่งให้ผลตามคาด

2
กรณีผิดถูกปฏิเสธ

บทที่ 5 สรุปและประเมินผล

ข้อจำกัดและแนวทางพัฒนาต่อ

⚠ ข้อจำกัด

ยังไม่มีส่วนติดต่อผู้ใช้

ทดสอบผ่าน SQL โดยตรง ยังไม่มีหน้าเว็บสำหรับลูกค้า

ข้อมูลใบขับขี่ยังไม่เข้ารหัส

license_no จัดเก็บเป็นข้อความธรรมดา ยังไม่ตรงตามความต้องการด้านความปลอดภัย

กฎ BR6 ยังตรวจสอบในแอปพลิเคชัน

ฐานข้อมูลยังไม่ปฏิเสธการจองรถที่มีสถานะ Maintenance โดยอัตโนมัติ

ค่าเช่ารวมยังบันทึกด้วยตนเอง

total_amount ยังไม่คำนวณจากจำนวนวันและค่าเช่าต่อวันโดยอัตโนมัติ

💡 แนวทางพัฒนาต่อ

พัฒนาเว็บแอปพลิเคชัน

ส่วนหน้า React เชื่อมต่อ Supabase ผ่าน RESTful API

เข้ารหัสข้อมูลส่วนบุคคล

ใช้ส่วนขยาย pgcrypto และกำหนด Row Level Security

ใช้ Trigger

ตรวจสอบกฎ BR6 คำนวณค่าเช่ารวม และปรับสถานะรถอัตโนมัติ

รายงานสำหรับผู้บริหาร

สร้าง View สรุปรายได้รายเดือนและอัตราการใช้รถ

ระบบเช่ารถยนต์ออนไลน์ (Online Car Rental System)

ขอบคุณครับ

ถาม–ตอบ (Q&A)

จัดทำโดย

นายสิทธิพันธุ์ ฮงเจริญ	68070182	บทนำ ทฤษฎี ความต้องการ และซอฟต์แวร์
นายจิราธิวัชร ทะวิลา	68070228	ลำดับการทำงาน กฎทางธุรกิจ และ ERD
นายภรัมพรหม เหลืองภุมรัตน์	68070229	Normalization พจนานุกรมข้อมูล DDL และข้อมูลตัวอย่าง
นายพูนพิพัฒน์ ช่างเกวียน	68070329	การทดสอบ SQL สรุปผล และถาม–ตอบ

6
ตารางในรูปแบบ 3NF

30
Constraints

19
คำสั่งทดสอบ

7
กฎทางธุรกิจ